SSCD (CO314) Assignment 2

Name : Raktim Ranjan Handique

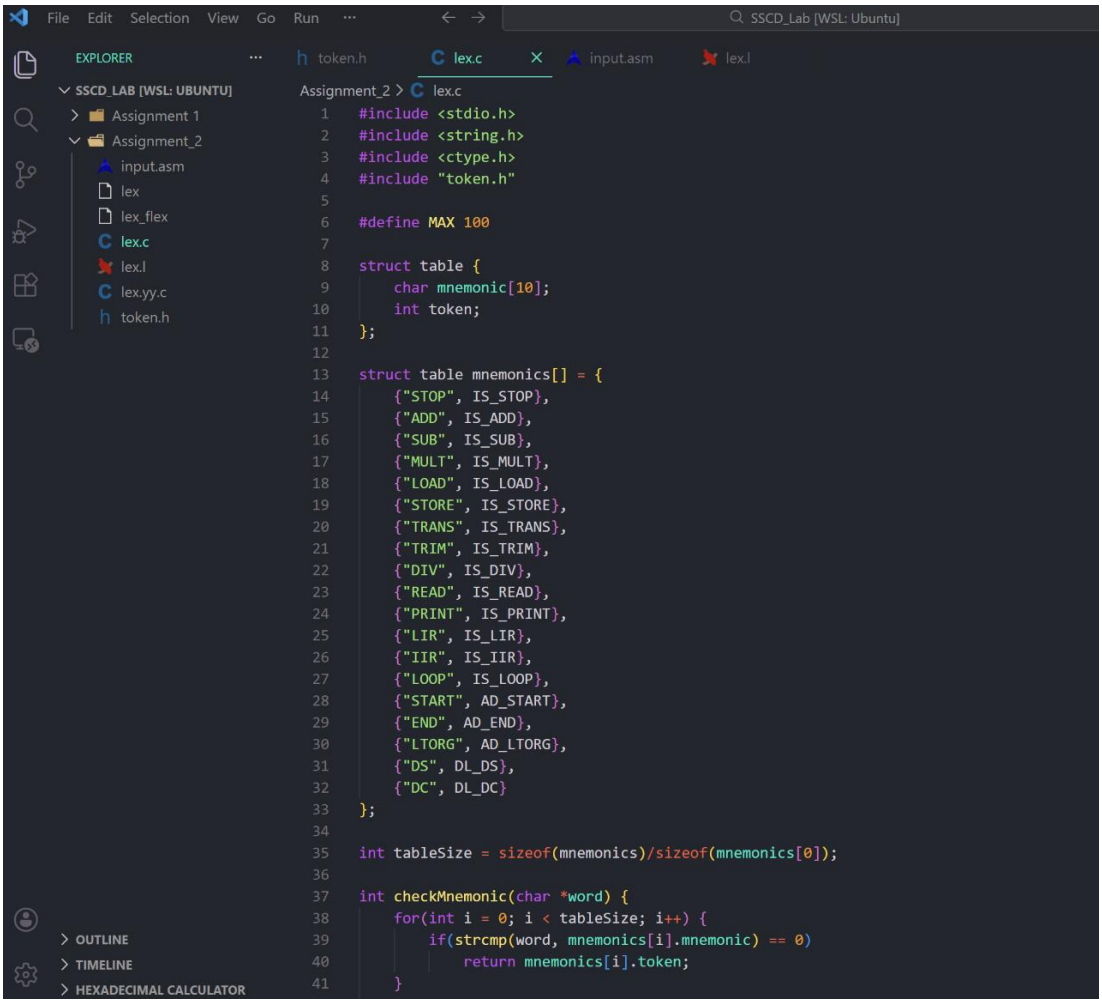
Roll No. CSB23017

Q1. Write a lexical analyser for the hypothetical assembly language provided herewith. The program should take as input a text file containing an assembly program and print the stream of token values corresponding to the items in the input file. For identifiers and numbers the actual input item should also appear within brackets along with the token value. e.g. the output may look like:

17 20(100) 21(AGAIN) 14 20(3) 21(TERM) ....

Ans:

a. lex.c file




```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include "token.h"

#define MAX 100

struct table {
    char mnemonic[10];
    int token;
};

struct table mnemonics[] = {
    {"STOP", IS_STOP},
    {"ADD", IS_ADD},
    {"SUB", IS_SUB},
    {"MULT", IS_MULT},
    {"LOAD", IS_LOAD},
    {"STORE", IS_STORE},
    {"TRANS", IS_TRANS},
    {"TRIM", IS_TRIM},
    {"DIV", IS_DIV},
    {"READ", IS_READ},
    {"PRINT", IS_PRINT},
    {"LIR", IS_LIR},
    {"IIR", IS_IIR},
    {"LOOP", IS_LOOP},
    {"START", AD_START},
    {"END", AD_END},
    {"LTORG", AD_LTORG},
    {"DS", DL_DS},
    {"DC", DL_DC}
};

int tableSize = sizeof(mnemonics)/sizeof(mnemonics[0]);

int checkMnemonic(char *word) {
    for(int i = 0; i < tableSize; i++) {
        if(strcmp(word, mnemonics[i].mnemonic) == 0)
            return mnemonics[i].token;
    }
```

File Edit Selection View Go Run ... SSCD_Lab [WSL: Ubuntu]

EXPLORER

SSCD_LAB [WSL: UBUNTU]
- Assignment 1
- Assignment_2
  - input.asm
  - lex
  - lex_flex
  - lex.c
  - lex.l
  - lex.yy.c
  - token.h

token.h | lex.c | input.asm | lex.l

Assignment_2 > lex.c

```c
int checkMnemonic(char *word) {
    return -1;
}

int main() {
    FILE *fp;
    char ch, buffer[MAX];
    int i = 0;

    fp = fopen("input.asm", "r");
    if(fp == NULL)
        return 0;

    while((ch = fgetc(fp)) != EOF) {

        if(isspace(ch))
            continue;

        if(isalpha(ch)) {
            buffer[i++] = ch;
            while(isalnum(ch = fgetc(fp)))
                buffer[i++] = ch;
            buffer[i] = '\0';
            i = 0;
            ungetc(ch, fp);

            int token = checkMnemonic(buffer);
            if(token != -1)
                printf("%d ", token);
            else
                printf("%d(%s) ", IDENTIFIER, buffer);
        }

        else if(isdigit(ch)) {
            buffer[i++] = ch;
            while(isdigit(ch = fgetc(fp)))
                buffer[i++] = ch;
            buffer[i] = '\0';
            i = 0;
            ungetc(ch, fp);
            printf("%d(%s) ", NUMBER, buffer);
```

OUTLINE
TIMELINE
HEXADECIMAL CALCULATOR

File Edit Selection View Go Run ... SSCD_Lab [WSL: Ubuntu]

EXPLORER

SSCD_LAB [WSL: UBUNTU]
- Assignment 1
- Assignment_2
  - input.asm
  - lex
  - lex_flex
  - lex.c
  - lex.l
  - lex.yy.c
  - token.h

token.h | lex.c | input.asm | lex.l

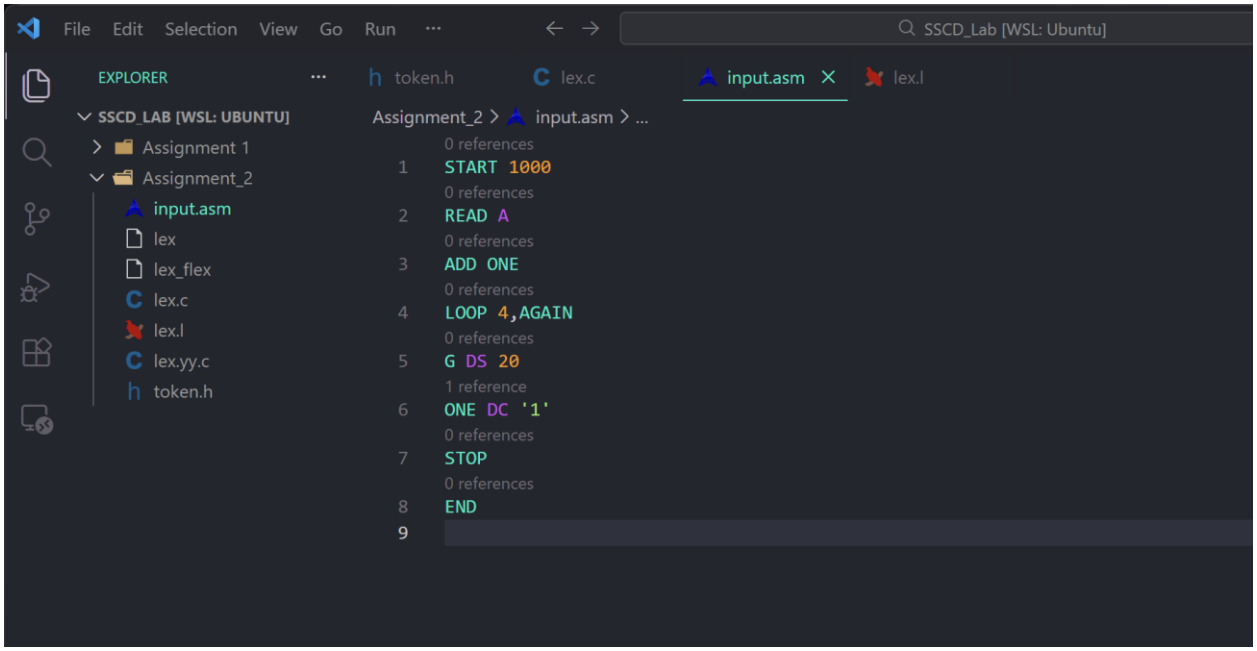
Assignment_2 > lex.c

```c
int main() {
    while((ch = fgetc(fp)) != EOF) {
        }

        else if(ch == '\'') {
            buffer[i++] = ch;
            while((ch = fgetc(fp)) != '\'')
                buffer[i++] = ch;
            buffer[i++] = ch;
            buffer[i] = '\0';
            i = 0;
            printf("%d(%s) ", STRING, buffer);
        }

        else if(ch == ',') {
            printf("%d ", COMMA);
        }
    }

    fclose(fp);
    return 0;
}
```

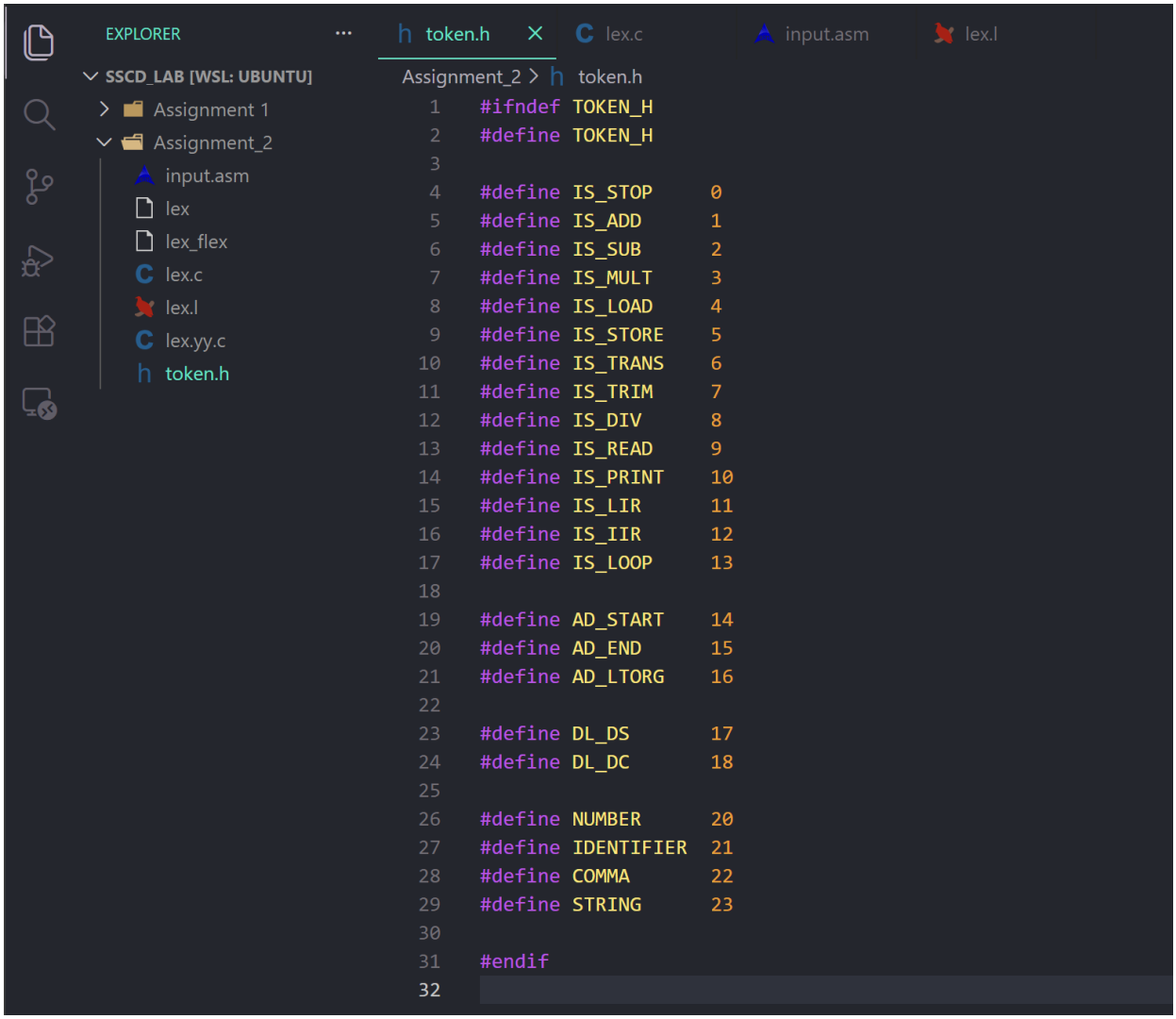
Output:

```
14 20(1000) 9 21(A) 1 21(ONE) 13 20(4) 22 21(AGAIN) 21(G) 17 20(20) 21(ONE) 18 23('1') 0 15
```

Input.asm file :



Token.h file :


EXPLORER
SSCD_LAB [WSL: UBUNTU]
Assignment 1
Assignment_2
input.asm
lex
lex_flex
lex.c
lex.l
lex.yy.c
token.h
token.h
lex.c
input.asm
lex.l
Assignment_2 > token.h
#ifndef TOKEN_H
#define TOKEN_H
#define IS_STOP 0
#define IS_ADD 1
#define IS_SUB 2
#define IS_MULT 3
#define IS_LOAD 4
#define IS_STORE 5
#define IS_TRANS 6
#define IS_TRIM 7
#define IS_DIV 8
#define IS_READ 9
#define IS_PRINT 10
#define IS_LIR 11
#define IS_IIR 12
#define IS_LOOP 13
#define AD_START 14
#define AD_END 15
#define AD_LTORG 16
#define DL_DS 17
#define DL_DC 18
#define NUMBER 20
#define IDENTIFIER 21
#define COMMA 22
#define STRING 23
#endif

Lex.l file :

```
%{
#include <stdio.h>
#include "token.h"
%}

%%

STOP        { printf("%d ", IS_STOP); }
ADD         { printf("%d ", IS_ADD); }
SUB         { printf("%d ", IS_SUB); }
MULT        { printf("%d ", IS_MULT); }
LOAD        { printf("%d ", IS_LOAD); }
STORE       { printf("%d ", IS_STORE); }
TRANS       { printf("%d ", IS_TRANS); }
TRIM        { printf("%d ", IS_TRIM); }
DIV         { printf("%d ", IS_DIV); }
READ        { printf("%d ", IS_READ); }
PRINT       { printf("%d ", IS_PRINT); }
LIR         { printf("%d ", IS_LIR); }
IIR         { printf("%d ", IS_IIR); }
LOOP        { printf("%d ", IS_LOOP); }

START       { printf("%d ", AD_START); }
END         { printf("%d ", AD_END); }
LTORG       { printf("%d ", AD_LTORG); }

DS          { printf("%d ", DL_DS); }
DC          { printf("%d ", DL_DC); }

[0-9]+              { printf("%d(%s) ", NUMBER, yytext); }
\'[^\']*\'          { printf("%d(%s) ", STRING, yytext); }
[A-Za-z][A-Za-z0-9]*    { printf("%d(%s) ", IDENTIFIER, yytext); }
","                 { printf("%d \n", COMMA); }

[ \t\n]             ;
.                   ;

%%

int main() {
    yylex();
    return 0;
}
```